

Slot 1 · Break your own service

File: `logs/silent_failure.md` · **Done when:** `uv run pytest tests/test_slot01.py` shows 4 passed · **Time:** 12 min

Today's idea

Normal software crashes when something is wrong. ML software usually does **not**. It keeps giving answers — wrong answers — with full confidence. Today you will see this with your own eyes, on your own laptop.

Step 1 · Start the service

```
make serve
curl -s localhost:8000/health
```

You should see:

```
up on :8000 (pid 12345)
{"status":"ok","model_loaded":true,"store_rows":900}
```

Stuck? If it says `port 8000 is already in use`, run the `kill <number>` it prints and try again.

Step 2 · Ask for three predictions

`scripts/predict.sh` sends one order to the service: 3 items, rainy evening. We save the answers in our write-up file as we go.

```
echo "## Before" > logs/silent_failure.md
scripts/predict.sh R0001 | tee -a logs/silent_failure.md
scripts/predict.sh R0042 | tee -a logs/silent_failure.md
scripts/predict.sh R0500 | tee -a logs/silent_failure.md
```

Look for `eta_minutes` and the last line, `HTTP 200`. You should get about:

restaurant	ETA before
R0001	49.0 min
R0042	54.0 min
R0500	37.0 min

These look sensible. Different restaurants, different answers.

Step 3 · An "upstream team" changes one thing

Imagine the maps team changed distance from **kilometres to metres** this morning. They did not rename the column. They did not tell you.

```
uv run python -m eta.corrupt units
curl -s -X POST localhost:8000/reload-store
```

We did not touch the model. We did not touch the service code.

Step 4 · Ask the same three questions again

```
echo "## After" >> logs/silent_failure.md
scripts/predict.sh R0001 | tee -a logs/silent_failure.md
scripts/predict.sh R0042 | tee -a logs/silent_failure.md
scripts/predict.sh R0500 | tee -a logs/silent_failure.md
```

You should get about **111.7**, **111.9** and **102.7** minutes. And still `HTTP 200`.

Now check the log:

```
scripts/serve.sh logs
```

Only normal `200 OK` lines. No error. No warning.

Think 1. What is the status code? 2. Did anything go in the log? 3. Before, the three answers were 17 minutes apart. Now they are only 9 minutes apart. Why did the model stop telling restaurants apart?

Step 5 · Finish the write-up, then undo the damage

```
echo "Nothing warned me. Still HTTP 200." >> logs/silent_failure.md
uv run pytest tests/test_slot01.py
uv run python -m eta.corrupt restore
curl -s -X POST localhost:8000/reload-store
```

The test should say **4 passed**. The last two lines put the data back to normal for the next slots.

Extra (if you finish early)

Try a different kind of damage — a column that suddenly becomes empty:

```
uv run python -m eta.corrupt nulls
curl -s -X POST localhost:8000/reload-store
scripts/predict.sh R0001
```

Still 200? Open `eta/service.py` and find the line `X = X.fillna(0)`. That line is why. Remember to run `restore` and `reload-store` again after.

If something goes wrong

what you see	what to do
ETA did not change after Step 3	You forgot <code>reload-store</code> . Run it and try again.
ETAs are strange even in Step 2	An old corruption is still there. Run <code>restore</code> , then <code>reload-store</code> .
<code>service</code> is not running	<code>make serve</code>
<code>used_default_profile: true</code>	Use a restaurant between R0001 and R0899.